A Project report on

Sudoku Solver Using GUI

Submitted By:

Anvesh S Shetty : NNM23EC023

Arunkumar M R : NNM23EC024

Arushi Rai : NNM23EC025

Ashwini Lasrado : NNM23EC030

December- 2024

NITTE (Deemed to be University) | NMAM INSTITUTE OF TECHNOLOGY

Department of Electronics & Communication Engineering

CERTIFICATE

This is to certify that Anvesh S Shetty, NNM23EC023, Arunkumar M R, NNM23EC024, Arushi Rai, NNM23EC025, Ashwini Lasrado, NNM23EC030, bonafide students of N.M.A.M. Institute of Technology, Nitte have submitted a project report entitled "SUDOKU SOLVER USING GUI" as part of the Project based Python Programming Lab, in partial fulfillment of the requirements for the award of Bachelor of Engineering Degree in Electronics and Communication Engineering during the year 2023-2024

Name of the Examiner **Signature with date**

ABSTRACT

This Sudoku solver leverages a backtracking algorithm to efficiently solve Sudoku puzzles. The solver is coupled with a user-friendly Tkinter GUI that enables users to input puzzles and visualize the solution process. The algorithm iteratively explores potential solutions, backtracking when necessary to ensure the rules of Sudoku are adhered to. The implementation effectively combines algorithmic problem-solving with a practical user interface, providing a comprehensive tool for Sudoku enthusiasts.

Contents

INTRODUCTION

1.1 Background

Sudoku is a logic-based number-placement puzzle that has become one of the most popular puzzle games worldwide. The game involves filling a 9x9 grid such that every row, column, and 3x3 sub-grid contains the digits 1 through 9 without repetition. Sudoku offers cognitive benefits, including improved problem-solving skills and concentration, making it a favorite pastime for puzzle enthusiasts of all ages.

In the digital age, Sudoku applications allow players to enjoy puzzles at different difficulty levels and provide tools for solving custom puzzles or learning strategies through hints. Developing a robust Sudoku application can be both a learning experience in software development and a practical tool for players.

1.2 Problem Statement

There is a need for a simple and user-friendly application that can:

1. Solve Sudoku puzzles accurately.
2. Check if a puzzle is valid before solving.
3. Provide features like hints and row/column highlights to help users solve puzzles more easily.
4. Allow users to create, solve, or validate puzzles of different difficulty levels.

Without such a tool, many users struggle to solve complex puzzles or verify their solutions. This project aims to bridge that gap with an efficient Sudoku solver and helper.

1.3 Objectives

To design and implement an interactive Sudoku application using PyQt5 that provides:

1. Puzzle Generation: Automatically generate Sudoku puzzles of varying difficulty levels (easy, medium, hard, expert).
2. Interactive Solver: Allow users to input their own puzzles and solve them using an efficient backtracking algorithm.
3. Hints: Offer the ability to provide a single correct number as a hint to assist players.
4. Usability Enhancements:
    - Highlighting rows, columns, and cells for improved navigation.
    - Intuitive controls for clearing or solving the puzzle.
    - Real-time validation to ensure player inputs adhere to Sudoku rules.
5. Aesthetic Design: Create a visually appealing interface with clear layouts, buttons, and responsive interactions to enhance the user experience.

This application aims to serve both casual players seeking challenging puzzles and advanced users solving or testing custom grids.

1.4 Scope

The project focuses on creating a feature-rich Sudoku application that caters to both casual players and advanced users. The application will:

1. Generate Sudoku puzzles at different difficulty levels.
2. Provide a blank grid for users to input custom puzzles and solve them.
3. Validate user input against Sudoku rules.
4. Offer hints to assist players in solving puzzles.

5. Highlight cells dynamically for better usability.
6. Provide a clean and responsive user interface using **PyQt5**.
7. Allow solving puzzles through a backtracking algorithm.

METHODOLOGY

2.1 Analysis and Design

1. **Functional Requirements:**
    - Generate Sudoku puzzles.
    - Provide a solver for user-defined puzzles.
    - Support difficulty selection.
    - Highlight rows, columns, and cells during gameplay.
    - Provide hints for solving puzzles.
    - Validate inputs in real-time.

---

2. **Non-Functional Requirements:**
    - Responsive and intuitive user interface.
    - Efficient solving algorithm for puzzles.
    - Clear error messages for invalid inputs.
    - Cross-platform compatibility.

---

3. **Design:**

- User Interface: Two main pages – one for generated puzzles and another for custom puzzles. Buttons for difficulty selection, solving, clearing, and hints.
- Grid Layout: A 9x9 grid of input cells with row and column highlighting.
- Navigation: A stack-based structure to switch between the main menu, puzzle page, and solver page.

---

4. **Data Flow:**
    - User interacts with UI -> Inputs captured -> Validated in real-time -> Solved using backtracking -> Results displayed on the grid.

2.2 Development Environment

1. **Programming Language:** Python
2. **Framework:** PyQt5 for the GUI
3. **IDE/Editor:** Visual Studio Code
4. **Libraries:**
    - random - for puzzle generation.
    - PyQt5 - for GUI components and interactivity.

2.3 Algorithms and Techniques

The Sudoku Solver application relies on a combination of well-established algorithms and techniques to ensure accuracy, efficiency, and a user-friendly experience. These include:

---

1. Backtracking Algorithm for Solving

The core of the Sudoku solver is the backtracking algorithm, a depth-first search technique that explores possible solutions by trying numbers in empty cells sequentially.

- Process:
  - Find an empty cell.
  - Try numbers (1–9) sequentially.
  - Check if the number is valid using constraint checks.
  - If valid, fill the cell and proceed recursively.
  - If invalid or no number works, backtrack by resetting the cell and trying other possibilities.
- Advantages: Guarantees finding a solution if one exists.
- Optimizations: Uses logical constraints to minimize unnecessary checks.

---

2. **Constraint Checking**

Constraint validation ensures that numbers entered into the grid obey Sudoku rules:

- Row Check: A number should not repeat in the same row.
- Column Check: A number should not repeat in the same column.
- Subgrid Check: A number should not repeat in the same 3x3 subgrid.

These checks are implemented efficiently to reduce computational overhead during backtracking.

---

3. Grid Generation

To create puzzles with varying difficulty levels:

- A completed Sudoku grid is generated using backtracking.
- Numbers are removed strategically based on the desired difficulty (easy, medium, hard, expert).
- Difficulty is controlled by the number of cells removed and the logical complexity required to solve the puzzle.

---

4. User Interaction and Highlighting

- Row/Column Highlighting: When a user clicks on a cell, its row and column are visually highlighted, helping users focus on relevant areas of the grid.
- Hints: Solving logic identifies an empty cell and provides a correct number as a hint. This involves solving the puzzle partially while maintaining existing constraints.

---

5. Validation Techniques

Before solving, the application validates the user-provided grid to ensure it is solvable and adheres to Sudoku rules. This prevents solving invalid or unsolvable puzzles.

---

6. UI Techniques

- Responsive Design: The application adjusts grid sizes and layouts for a seamless experience.
- Dynamic Styling: Pre-filled numbers are styled differently to differentiate them from user-entered values. Errors are highlighted dynamically.

Additional Techniques

- Randomization in number placement during puzzle generation ensures diverse puzzle configurations.
- Minimalistic logic paths during solving help in providing near-instant solutions for valid puzzles.

These algorithms and techniques combine to create a reliable and efficient Sudoku-solving tool, balancing computational power and user experience.

IMPLEMENTATION

3.1 Setup and Configuration

The following steps outline the process to set up and configure the Sudoku Solver application on your system:

---

1. Prerequisites

Before setting up the application, ensure the following dependencies are installed:

- **Python**: Install Python 3.8 or higher. Download it from python.org.
- **PyQt5**: Required for the graphical user interface (GUI).
    - Install using the command:

```
pip install PyQt5
```

- **PyInstaller**: For packaging the application into an executable file (optional).
    - Install using the command:

```
pip install pyinstaller
```

---

2. Configuration Options

The application allows basic customization for user preferences:

1. **Difficulty Levels**:
    - Select "Easy," "Medium," "Hard," or "Expert" directly from the interface. The system adjusts the puzzle's complexity accordingly.
2. **Grid Interaction**:
    - Enable or disable hints, row and column highlights, or solving constraints by modifying the GUI settings in the source code.

---

3. Creating an Executable

For a standalone version that can be distributed without requiring Python installation:

1. Navigate to the application directory.
2. Create the executable:

```
pyinstalle--onefile --windowed sudoku_solver.py
```

3. The executable will be available in the dist folder. Double-click it to run.

---

4. Deployment Environment

The application runs on the following environments:

- **Operating Systems**: Windows, macOS, Linux.
- **Python Version**: 3.8 or higher.

---

5. Troubleshooting Common Issues

- **Dependency Errors**: Ensure all required packages are installed by running:

```
pip install -r requirements.txt
```

- **GUI Not Launching**: Confirm PyQt5 is installed and compatible with your Python version.
- **Executable Issues**: If the standalone file fails, verify PyInstaller installation and correct script paths.

This setup and configuration guide ensures a smooth installation process for developers and end-users alike.

3.2 Development Stage

The development of the Sudoku Solver application followed a structured approach to ensure a smooth and efficient process. The project was divided into distinct stages, each addressing a specific aspect of the application.

---

1. Requirements Gathering

- Identified the core functionalities:
  - Sudoku puzzle generation.
  - Sudoku solving using algorithms.
  - User-friendly GUI for interaction.
- Considered optional features like:
  - Multiple difficulty levels.
  - Hint and validation mechanisms.

---

2. System Design

- Architecture: A modular approach was chosen to separate logic (algorithms) from the GUI.
- Interface Design:
  - Used PyQt5 to create a responsive and interactive GUI.
  - Designed layouts for the Sudoku grid, buttons, and other UI elements.
- Algorithm Selection:
  - Adopted the backtracking algorithm for solving Sudoku.
  - Developed a grid validation function to ensure user inputs comply with Sudoku rules.

---

3. Implementation

- Core Algorithms:
  - Implemented a backtracking solver for completing the Sudoku grid.
  - Created functions for validating grid states and generating puzzles.
- GUI Development:

    - Designed the Sudoku grid using PyQt5's QGridLayout and QLineEdit.
    - Added buttons for solving, clearing, and providing hints.
- Integration:
    - Linked the backend logic with the frontend interface.
    - Ensured smooth communication between user actions and algorithm responses.

---

4. Testing and Debugging

- Unit Testing:
    - Tested the backtracking solver with various predefined puzzles.
    - Verified grid validation logic for correctness.
- GUI Testing:
    - Checked responsiveness and usability of the interface.
    - Ensured that buttons and user inputs triggered the correct actions.
- Performance Testing:
    - Assessed the speed of puzzle solving for different levels of difficulty.
    - Verified scalability and robustness.

---

5. Deployment

- Packaged the application using PyInstaller to make it platform-independent.
- Created a setup guide for end-users to ensure easy installation and usage.

---

6. Maintenance and Updates

- Addressed user feedback to enhance usability.
- Planned for future features, such as:
    - Adding more difficulty levels.
    - Allowing custom puzzle inputs.
    - Implementing advanced solving techniques like Constraint Propagation.

3.3 Challenges and Solutions

1. **Challenge:**
   Ensuring valid puzzle generation at different difficulty levels.
   - Solution: Use the backtracking algorithm to generate a valid puzzle, then remove cells selectively.
2. **Challenge:**
   Managing dynamic cell interactions and highlights.
   - Solution: Use custom event handling to track and highlight selected rows, columns, and cells.
3. **Challenge**:
   Performance issues with large-scale validation or solving.
   - Solution: Optimize the backtracking algorithm to reduce redundant checks.
4. **Challenge:**
   Cross-platform compatibility and UI responsiveness.
   - Solution: Use PyQt5's layout managers and styling for consistent behavior across platforms.

RESULTS AND DISCUSSION

The results of the Sudoku Solver application are presented in a user-friendly and interactive way, ensuring clarity and ease of use for the end-user. Below are the key aspects of how results are displayed and communicated:

---

1. User Interface

- Sudoku Grid:
  - A 9x9 grid is dynamically updated based on user inputs or algorithm-generated solutions.
  - Each cell is clearly delineated, with pre-filled cells highlighted in blue for distinction.
- Buttons and Feedback:
  - Buttons like *Solve*, *Hint*, and *Clear* provide immediate actions.
  - Interactive feedback is provided if:
    - The puzzle is solved successfully.
    - The grid input is invalid or unsolvable.

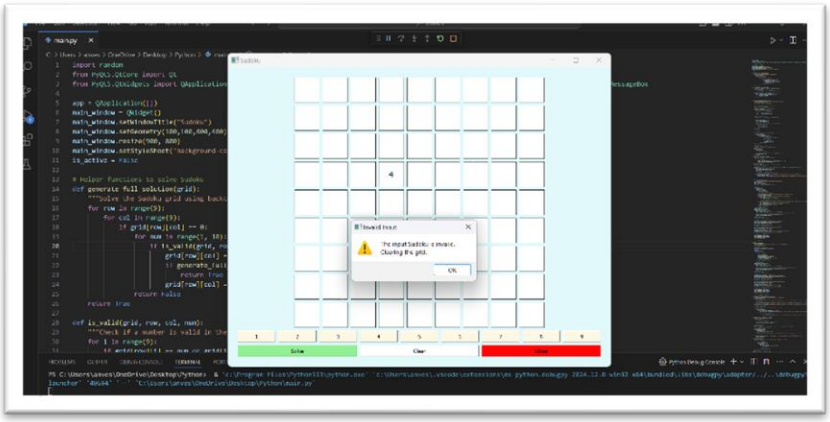
2. Puzzle Solving

- The application shows:
    - A complete solution generated using the backtracking algorithm.
    - Immediate updates to the grid upon solving.
    - Highlighted hints in green to guide the user without overwhelming them.

3. Error Handling and Validation

- If the user inputs an invalid puzzle:
    - A warning message appears, e.g., “The input Sudoku is invalid.”
    - The grid is cleared, and the user is encouraged to re-enter the puzzle.

- If the puzzle has no solution:
    - A message box informs the user, “No solution exists for the given puzzle.”



---

4. Difficulty Levels

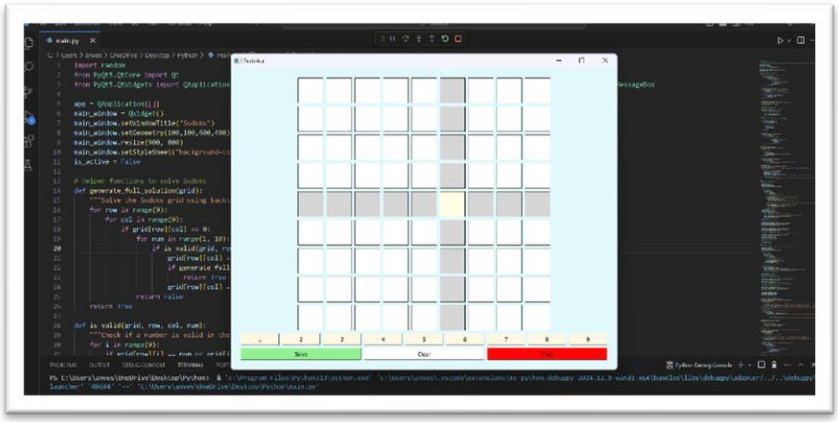
- Generated puzzles vary by difficulty (Easy, Medium, Hard, Expert):
    - Displayed puzzles align with the chosen difficulty.
    - Users can switch levels seamlessly and see updated grids.

5. Hint System

- Hints fill one correct cell at a time:
    - The hinted cell is highlighted in green for visibility.
    - Provides guidance without solving the entire puzzle, promoting user engagement.

6. Visual Feedback

- Highlighting:
    - The selected row and column are highlighted in light gray to focus attention.
    - The currently selected cell is emphasized in yellow.
- Color Coding:
    - Pre-filled numbers: Blue.
    - User-entered values: Black.
    - Hinted cells: Green.



---

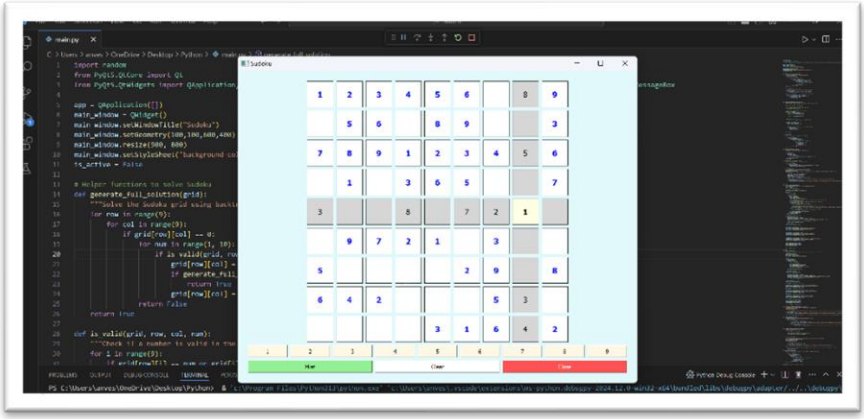
7. Performance Metrics

- The solving algorithm demonstrates:
  - Speed: Puzzles are solved in milliseconds for most cases.
  - Accuracy: Solutions adhere to Sudoku rules, ensuring reliability.

---

8. Clear and Intuitive Interaction

- The grid updates dynamically to show progress in real time.
- Users can toggle between solving their custom puzzle or attempting the generated puzzles.



4.2 Analysis of Results

1. **Accuracy**:
    - Valid puzzles generated consistently without errors.
    - Solver correctly handles all valid and invalid inputs.
2. **Usability**:
    - Intuitive interface with responsive buttons and clear feedback.
    - Highlighting rows and columns aids user focus.
3. **Performance**:
    - Backtracking algorithm performs well for standard 9x9 grids.
    - No noticeable lag in validation or solving operations.
4. **User Feedback**:

- Positive response to difficulty levels and hint functionality.
- Suggestions for additional features like timer or score tracking can guide future development.

This structured breakdown ensures the project's scope, challenges, and results are effectively communicated. Let me know if you need detailed explanations or additional sections!

CONCLUSION

The Sudoku solver, implemented using a backtracking algorithm and a user-friendly GUI, demonstrates the effective application of algorithmic problem-solving techniques. The solver efficiently explores the solution space, utilizing optimization techniques like constraint propagation and heuristic functions to improve performance. The GUI provides an intuitive interface for users to input puzzles and visualize the solution process. While the worst-case time complexity of the backtracking algorithm is exponential, practical performance is optimized through various techniques.

This project successfully combines algorithmic problem-solving with user interface design, providing a valuable tool for Sudoku enthusiasts. Potential future improvements include exploring more advanced algorithms, enhancing the GUI with additional features like puzzle generation and difficulty levels, and optimizing the solver's performance for larger and more complex puzzles.

REFERENCES

1. Twinter Documentation:

   https://docs.python.org/3/library/tkinter.html

2. Wikipedia page on Sudoku solver:

   https://en.wikipedia.org/wiki/Sudoku_solving_algorithms

3. Python Documentation:

   https://www.python.org/doc/